

UNIVERSIDAD DE LA HABANA

Facultad de Matemática y Computación



Proyecto Final de Inteligencia Artificial

Segmentación Óptima de Contenido

Autor: Lidier Robaina Caraballo

Grupo: C-412

mayo de 2026

Índice general

1. Introducción	1
2. Diseño e Implementación	2
2.1. Modelado	2
2.2. Solución exacta	3
2.3. Problema	3
2.3.1. Segmentación de Contenido	4
2.4. Metaheurísticas	4
2.4.1. Metaheurísticas de trayectoria	5
2.4.2. Metaheurísticas poblacionales	6
2.5. Integración del LLM	8
2.6. Dataset	9
3. Experimentos y Resultados	10
3.1. Experimento 1: Ajuste de la función objetivo	10
3.1.1. Caso 1: Segmentación alta	11
3.1.2. Caso 2: Segmentación media	12
3.1.3. Caso 3: Segmentación baja	13
3.1.4. Análisis	14
3.2. Experimento 2: Ajuste de las metaheurísticas	15
3.2.1. Ascenso de Colina	15
3.2.2. Recocido Simulado	16
3.2.3. Algoritmo Genético	18
3.2.4. Enjambre de Partículas	19
3.2.5. Análisis	21
3.3. Experimento 3: Evaluación de las metaheurísticas	21
3.3.1. Resultados	21
3.3.2. Análisis	23
4. Conclusiones	24
5. Recomendaciones	26

Capítulo 1

Introducción

La segmentación de contenido es una tarea fundamental en la organización automática de información: dividir una secuencia de elementos (párrafos, oraciones, fragmentos de vídeo) en bloques contiguos que presenten coherencia temática interna. Una segmentación de calidad permite mejorar la navegación en documentos extensos, la generación de resúmenes y la extracción de información. Tradicionalmente, la tarea se ha abordado con métodos basados en similitud léxica o modelos probabilísticos [1]; sin embargo, la aparición de grandes modelos de lenguaje (LLM) ofrece la posibilidad de evaluar la coherencia semántica de forma mucho más precisa, actuando como un oráculo que puntúa la cohesión de un segmento.

En este proyecto se trata la segmentación como un problema de optimización combinatoria: encontrar la partición de una secuencia que maximiza una función de coherencia agregada, evaluada mediante un LLM. Este enfoque permite estudiar cómo distintas estrategias algorítmicas pueden encontrar soluciones de alta calidad gestionando eficientemente un recurso costoso y limitado.

Se plantean los siguientes objetivos:

- Formalizar el problema de segmentación óptima basada en coherencia.
- Implementar una solución exacta que sirva como referencia para ajustar parámetros del problema.
- Desarrollar metaheurísticas que exploren el espacio de segmentaciones utilizando al LLM como función de coste.
- Diseñar un mecanismo de interacción con el LLM que incluya un *prompt* robusto, caché de evaluaciones y estrategias para reducir el número de llamadas.
- Construir un conjunto de instancias de prueba que permitan ajustar y analizar el comportamiento de los algoritmos.
- Comparar experimentalmente las configuraciones propuestas en términos de calidad de la solución, número de consultas al LLM y tiempo de ejecución.

Capítulo 2

Diseño e Implementación

2.1 Modelado

Sea la entrada del problema $S = (e_1, e_2, \dots, e_N)$, una secuencia finita de N elementos textuales. Se desea obtener una segmentación que maximice la suma de las coherencias individuales, sujeta a que los segmentos sean disjuntos, cubran completamente S y respeten el orden original.

Con este fin, se define una segmentación como un vector binario $\mathbf{x} \in \{0, 1\}^{N-1}$, donde $x_i = 1$ indica que existe un corte entre e_i y e_{i+1} . El número de segmentos es $k = 1 + \sum x_i$. Sea la función objetivo:

$$f(\mathbf{x}) = \sum_{j=1}^k \phi(s_j)$$

donde $\phi(s)$ es la puntuación de coherencia del segmento s devuelta por el LLM; la salida del problema es la segmentación $\mathbf{x}$ que maximiza f .

Sin embargo, por la naturaleza del dominio y la función objetivo anterior, la solución para cualquier instancia sería la solución degenerada de cortar cada elemento. Por tanto, para capturar el coste de generar más segmentos y penalizar segmentaciones excesivamente fragmentadas, la función objetivo final incorpora un término de penalización α sobre el número de segmentos:

$$f(\mathbf{x}) = \sum_{j=1}^k \phi(s_j) - \alpha \cdot k$$

Adicionalmente, los segmentos de longitud uno (en lo adelante, segmentos unitarios) reciben una puntuación fija ϕ_u . Esta decisión responde a dos razones:

1. **Eficiencia:** evaluar un segmento de un solo elemento con el LLM sería tan costoso como evaluar un segmento más largo, pero aportaría poca información relevante (la coherencia de un elemento consigo mismo es trivialmente máxima). Asignar un valor fijo evita llamadas innecesarias a la API.

2. **Control de la fragmentación:** al fijar ϕ_u podemos ajustar el equilibrio entre coherencia intrasegmental y parsimonia de la partición. Un valor demasiado alto favorecería segmentos unitarios; un valor bajo los desalienta.

El problema es de naturaleza combinatoria: si se permite cualquier número de puntos de corte, el espacio de soluciones crece exponencialmente (2^{N-1}). Aunque existe una solución exacta mediante programación dinámica, el coste de una evaluación de coherencia con llamada al LLM es elevado (latencia de red, consumo de API, límites de tasa) y este enfoque resulta inviable incluso para instancias de tamaño moderado. Por tanto, se investigan metaheurísticas que, mediante un número limitado de evaluaciones del LLM (del orden de cientos o unos pocos millares), sean capaces de aproximar soluciones de alta calidad. Estas metaheurísticas permiten explorar el espacio de soluciones sin necesidad de evaluar exhaustivamente todas las posibles segmentaciones.

2.2 Solución exacta

El módulo `dp.py` implementa un algoritmo de programación dinámica (DP) que calcula la segmentación óptima. Se define $dp[i]$ como la máxima puntuación alcanzable para los primeros i elementos. La recurrencia es:

$$dp[i] = \max_{0 \leq j < i} (dp[j] + \phi(e_{j+1} \dots e_i) - \alpha)$$

con $dp[0] = 0$. La solución se reconstruye almacenando los puntos de corte óptimos.

El algoritmo tiene complejidad $O(N^2 \cdot C)$, donde C es el coste de una evaluación de coherencia. Se utiliza para ajustar los parámetros de la función objetivo (α y ϕ_u) cuando las instancias son lo bastante pequeñas para evaluar exhaustivamente.

2.3 Problema

Uno de los principios fundamentales en el diseño de metaheurísticas es la separación estricta entre la lógica del problema y los algoritmos de búsqueda. Para ello, en el módulo `problem.py` se ha definido la clase abstracta `Problem`, que actúa como interfaz común para cualquier dominio de optimización. Esta clase encapsula todo el conocimiento específico del problema y obliga a que las metaheurísticas permanezcan completamente agnósticas del dominio.

La clase `Problem` declara los siguientes métodos, que toda subclase debe implementar según las necesidades de los algoritmos en que se vaya a utilizar:

- `create_solution()` – método abstracto que genera una solución factible, que puede ser aleatoria o construida mediante una heurística.

- `objective_function(solution)` – método abstracto que evalúa la calidad de una solución. Este *framework* asume que valores más altos indican mejores soluciones (maximización). Los problemas de minimización pueden adaptarse cambiando el signo de la función objetivo.
- `get_neighbor(solution, k)` – método requerido por las metaheurísticas de trayectoria. Genera hasta k vecinos y retorna el mejor de ellos junto con su *fitness*.
- `crossover(p1, p2)` – método requerido por los algoritmos genéticos. Crea dos hijos a partir de dos padres.
- `mutate(solution)` – método requerido por los algoritmos genéticos. Aplica una perturbación aleatoria sobre la solución.

2.3.1. Segmentación de Contenido

La clase `ContentSegmentationProblem` implementa la interfaz `Problem` para el problema formulado en la sección 2.1.

Generación de vecinos. Para la búsqueda local se exploran varias soluciones candidatas y se devuelve la mejor encontrada. En cada intento se elige aleatoriamente una de tres operaciones elementales:

- `add_cut`: convierte un 0 en 1.
- `remove_cut`: convierte un 1 en 0.
- `move_cut`: cambia un 1 por un 0 en un lugar y un 0 por un 1 en otro.

Estas operaciones garantizan que el vecindario explorado contenga soluciones estructuralmente cercanas y que todas las transiciones sean alcanzables en pocos pasos.

Cruce y mutación. El cruce es de un punto: se elige una posición aleatoria entre los bits de corte y se intercambian las colas de los dos padres. La mutación modifica cada bit de forma independiente con probabilidad $1/(N - 1)$, garantizando además que al menos un bit cambie para evitar la inmutabilidad de la población.

2.4 Metaheurísticas

En `metaheuristics.py` se implementa una jerarquía de clases de metaheurísticas basada en la taxonomía y algoritmos propuestos en [2]. Se define una clase base `Metaheuristic` que expone el método `solve(problem: Problem)` y dos plantillas que heredan de ella: una para algoritmos de trayectoria y otra para algoritmos poblacionales. Esta arquitectura ha sido validada implementando dos metaheurísticas concretas de cada tipo, lo que demuestra su capacidad de abstracción y flexibilidad.

Ambas plantillas encapsulan la gestión de métricas (evaluaciones, generaciones, historial) en un diccionario de estado común e incorporan un mecanismo de terminación unificado basado en `max.evaluations`. De este modo, este *framework* permite desarrollar cualquier metaheurística sin necesidad de modificar la lógica de la plantilla ni sobrescribir el método `solve`: basta con especificar sus parámetros y operadores particulares.

2.4.1. Metaheurísticas de trayectoria

La plantilla `SingleSolutionMetaheuristic` encapsula el bucle de búsqueda en el método `solve`. Cada algoritmo concreto se limita a implementar los métodos abstractos `generate` y `accept` que capturan el comportamiento específico de exploración y explotación.

```
def solve(problem):
    current = problem.create_solution()
    current_fit = problem.objective_function(current)
    best, best_fit = current, current_fit
    while not termination():
        candidate, cand_fit = generate(current)
        if accept(candidate, cand_fit, current, current_fit):
            current, current_fit = candidate, cand_fit
        if current_fit > best_fit:
            best, best_fit = current, current_fit
    return best, best_fit
```

Ascenso de colina (Hill Climbing)

Parámetros:

- `stagnation_limit`: iteraciones consecutivas sin mejora que detienen la búsqueda.
- `neighborhood_size`: número de vecinos generados en cada paso.

El algoritmo explora la vecindad en cada iteración y se mueve únicamente si encuentra una mejora estricta. Un `neighborhood_size` mayor amplía la exploración local a costa de más evaluaciones, mientras que el `stagnation_limit` evita un gasto excesivo de recursos cuando se ha alcanzado un óptimo local.

Recocido simulado (Simulated Annealing)

Parámetros:

- `T0`: temperatura inicial; valores altos favorecen la aceptación de soluciones peores al comienzo.
- `alpha`: factor de enfriamiento ($0 < \alpha < 1$).

- `iters_per_temp`: iteraciones antes de reducir la temperatura.

El recocido simula el enfriamiento de un material; la temperatura T controla la probabilidad de aceptar soluciones peores, permitiendo escapar de óptimos locales. En cada paso se genera un vecino aleatorio y se acepta según el criterio de Metropolis:

$$\text{accept(candidate)} = \begin{cases} \text{True} & \text{si } \Delta f \geq 0 \quad \text{o} \quad \text{random}(0,1) < e^{-\frac{\Delta f}{T}}, \\ \text{False} & \text{en otro caso,} \end{cases}$$

donde $\Delta f = \text{cand_fit} - \text{current_fit}$ y T es la temperatura actual.

Cada `iters_per_temp` iteraciones la temperatura se enfría según $T \leftarrow \alpha \cdot T$, reduciendo progresivamente la probabilidad de aceptar soluciones peores. El proceso se detiene cuando $T \leq T_{\text{mín}}$.

2.4.2. Metaheurísticas poblacionales

La plantilla `PopulationMetaheuristic` define el flujo evolutivo común de la población de soluciones mediante el método `solve`, que orquesta las etapas de inicialización, selección, reproducción y reemplazo. Las subclases concretas implementan los métodos abstractos (`initialize_population`, `select`, `breed` y `replace`).

```
def solve(problem):
    population, fitness = initialize_population()
    while not termination():
        parents = select(population, fitness)
        children = breed(parents)
        children_fitness = [problem.objective_function(c) for c in children]
        population, fitness = replace(population, fitness,
                                      children, children_fitness)
        best, best_fitness = update_best(population, best)
    return best, best_fitness
```

Algoritmo Genético

Parámetros:

- `population_size`: número de individuos en la población.
- `mutation_rate`: probabilidad de mutar cada bit de un hijo.
- `elitism`: cantidad de mejores individuos que pasan intactos a la siguiente generación.
- `max_generations`: número máximo de generaciones.

El algoritmo mantiene una población de soluciones que evoluciona mediante selección, cruce y mutación. La selección por torneo binario (elige el mejor entre dos individuos aleatorios) favorece a los más aptos. El cruce de un punto combina dos padres y la mutación bit-flip introduce diversidad. El elitismo garantiza que las mejores soluciones no se pierdan.

Enjambre de partículas (Particle Swarm Optimization, PSO)

Se implementa una versión binaria. Cada partícula i posee:

- Posición: $\mathbf{x}_i \in \{0, 1\}^{N-1}$,
- Velocidad: $\mathbf{v}_i \in \mathbb{R}^{N-1}$,
- Mejor posición personal: $\mathbf{p}_i$,
- Mejor posición global: $\mathbf{g}$.

Parámetros:

- w : inercia; pondera la velocidad anterior (balance exploración-explotación).
- c_1 : coeficiente cognitivo, atracción hacia la mejor posición personal $\mathbf{p}_i$.
- c_2 : coeficiente social, atracción hacia la mejor posición global $\mathbf{g}$.
- v_max : límite máximo para cada componente de velocidad; evita divergencia y saturación de la sigmoide.

Las partículas se mueven en el espacio de búsqueda combinando su propia experiencia ($\mathbf{p}_i$) y la del enjambre ($\mathbf{g}$). La actualización de velocidad sigue la ecuación clásica:

$$v_i^{(t+1)} = w \cdot v_i^{(t)} + c_1 r_1 (p_i - x_i^{(t)}) + c_2 r_2 (g - x_i^{(t)}),$$

donde $r_1, r_2 \sim \mathcal{U}(0, 1)$. La velocidad se limita al intervalo $[-v_{\max}, v_{\max}]$ y la posición binaria se obtiene aplicando una función sigmoide:

$$P(x_i^{(t+1)} = 1) = \frac{1}{1 + e^{-v_i^{(t+1)}}},$$

$$x_i^{(t+1)} = \begin{cases} 1 & \text{si } \text{random}(0, 1) < P(x_i^{(t+1)} = 1), \\ 0 & \text{en otro caso.} \end{cases}$$

En la adaptación a los métodos de la plantilla poblacional:

- **initialize_population**: genera posiciones aleatorias y velocidades en $[-v_{\max}, v_{\max}]$.
- **select**: devuelve la población completa, pues PSO no descarta individuos.
- **breed**: aplica las ecuaciones de actualización a todas las partículas.
- **replace**: asigna las nuevas posiciones como población actual y actualiza los mejores personales y global.

2.5 Integración del LLM

La evaluación de la coherencia semántica de los segmentos se implementa en el módulo `llm.py` y constituye el núcleo de la función objetivo. Para cada segmento identificado por la metaheurística, el sistema decide si es necesario consultar al modelo de lenguaje o si puede emplearse un valor predefinido, y en caso de consulta, gestiona la comunicación con la API, el almacenamiento en caché y la tolerancia a fallos.

La evaluación se apoya en la API de Google GenAI, utilizando el modelo `gemini-3.1-flash-lite`. Puesto que las claves de API son información sensible que no debe publicarse en repositorios, cada usuario debe proporcionar sus propias credenciales en un archivo `keys.txt` ubicado en la raíz del proyecto, donde cada línea contiene una clave válida. La rotación entre claves se activa exclusivamente cuando se recibe un error 429 (Resource Exhausted), lo que permite continuar las consultas sin interrupción y sin intervención manual.

La función `get_coherence` sigue el siguiente procedimiento:

- Si el segmento está formado por un único elemento textual, se devuelve el valor fijo ϕ_u .
- Para segmentos de longitud mayor que uno, se construye el texto concatenando los fragmentos correspondientes y se comprueba si ya existe una entrada en la caché persistente (`data/llm.cache.json`). En caso afirmativo, se retorna el valor almacenado.
- Si el segmento no está en caché, se elabora un *prompt* en español que solicita explícitamente una puntuación numérica entre 0 y 1. El *prompt* utilizado es:

Evalúa la coherencia semántica del siguiente fragmento de texto en una escala de 0 a 1, donde 0 es completamente incoherente y 1 es perfectamente coherente. Responde ÚNICAMENTE con un número decimal entre 0 y 1.

Texto: `segment_text`

Coherencia:

La respuesta se guarda inmediatamente en el archivo de caché, garantizando que futuras evaluaciones del mismo segmento no requieran nuevas consultas.

El uso de caché persistente es esencial para amortizar el coste computacional y económico de las evaluaciones repetidas durante la ejecución de las metaheurísticas, y para garantizar que la comparación entre distintas configuraciones se realice sobre puntuaciones idénticas para los mismos segmentos.

2.6 Dataset

Se dispone de un conjunto de 8 instancias etiquetadas, cada una compuesta por una lista de 20 fragmentos textuales y su segmentación óptima asociada. Las instancias se utilizan de la siguiente manera:

- **Instancias de validación (3 primeras):** Se emplean para ajustar los parámetros de la función objetivo (α , ϕ_u) y los hiperparámetros de las metaheurísticas.
- **Instancias de prueba (5 restantes):** Se reservan para la evaluación final del rendimiento de las metaheurísticas con sus mejores configuraciones.

Las secuencias textuales y sus correspondientes segmentaciones óptimas fueron generadas por el modelo de lenguaje DeepSeek, solicitándole explícitamente fragmentos con cambios temáticos internos que dieran lugar a segmentos naturales de diferente longitud y cohesión semántica, y que además proporcionara la división correcta. De esta forma se obtuvieron colecciones de 20 fragmentos con entre 2 y 12 segmentos implícitos, junto con la segmentación de referencia. Posteriormente, dos anotadores humanos independientes revisaron cada instancia para confirmar que la segmentación propuesta por el modelo reflejaba fielmente la estructura lógica del contenido. Este proceso de verificación garantiza que las etiquetas de referencia sean fiables y que las instancias capturen de manera realista la tarea de segmentación.

A pesar de su tamaño reducido, el conjunto de instancias abarca una variedad suficiente de casos (segmentos unitarios, segmentos largos, transiciones abruptas y difusas) para poner a prueba tanto la función de evaluación como los distintos comportamientos de búsqueda de las metaheurísticas. La presencia de las segmentaciones de referencia permite medir de forma objetiva la calidad de las soluciones encontradas, y la división en validación y prueba asegura que los hiperparámetros no se sobreajusten a los datos de evaluación.

Idealmente se dispondría de un número mucho mayor de instancias y de secuencias de texto más extensas, lo que permitiría una evaluación más robusta y una mayor confianza en las conclusiones. Sin embargo, la dependencia del sistema de un LLM externo, junto con las restricciones operativas del entorno de desarrollo que imponen límites prácticos al número de llamadas a la API y a la duración de las ejecuciones, impidió experimentar con conjuntos de datos de mayor escala. Aun así, el dataset actual cumple con el propósito de validar la arquitectura del *framework* y comparar el desempeño relativo de las metaheurísticas implementadas.

Capítulo 3

Experimentos y Resultados

Métrica de evaluación. Para medir la calidad de una segmentación propuesta frente a la segmentación de referencia se utiliza la medida F_1 , habitual en tareas de clasificación binaria. Dada una solución como vector de cortes $\mathbf{c} \in \{0, 1\}^{N-1}$, se definen:

- Verdaderos positivos (VP): cortes que aparecen tanto en la solución como en la referencia.
- Falsos positivos (FP): cortes predichos que no están en la referencia.
- Falsos negativos (FN): cortes de la referencia omitidos en la solución.

La precisión es $P = VP/(VP + FP)$, la exhaustividad $R = VP/(VP + FN)$, y la puntuación F_1 se calcula como su media armónica:

$$F_1 = 2 \cdot \frac{P \cdot R}{P + R},$$

asignando $F_1 = 0$ cuando no hay ningún corte real ni predicho. Esta métrica resume en un solo valor el equilibrio entre la capacidad de detectar los cortes correctos y la de evitar cortes espurios, lo que la hace especialmente adecuada para comparar soluciones de segmentación.

3.1 Experimento 1: Ajuste de la función objetivo

Objetivo. Determinar los valores óptimos de los parámetros de la función objetivo: la coherencia unitaria ϕ_u (valor asignado a segmentos de un solo elemento) y el factor de penalización α por cada segmento.

Metodología. Se realizó una búsqueda en cuadrícula variando $\phi_u \in [0, 1]$ con paso 0,1 y $\alpha \in [0, 1]$ con paso 0,1. Para cada combinación se ejecutó un algoritmo de programación dinámica (solución exacta) sobre las tres instancias de validación, calculando el F_1 respecto a los cortes reales. Se seleccionaron los parámetros que maximizaron el F_1 promedio.

3.1.1. Caso 1: Segmentación alta

Esta instancia presenta una segmentación fina, con muchos cortes y numerosos segmentos unitarios.

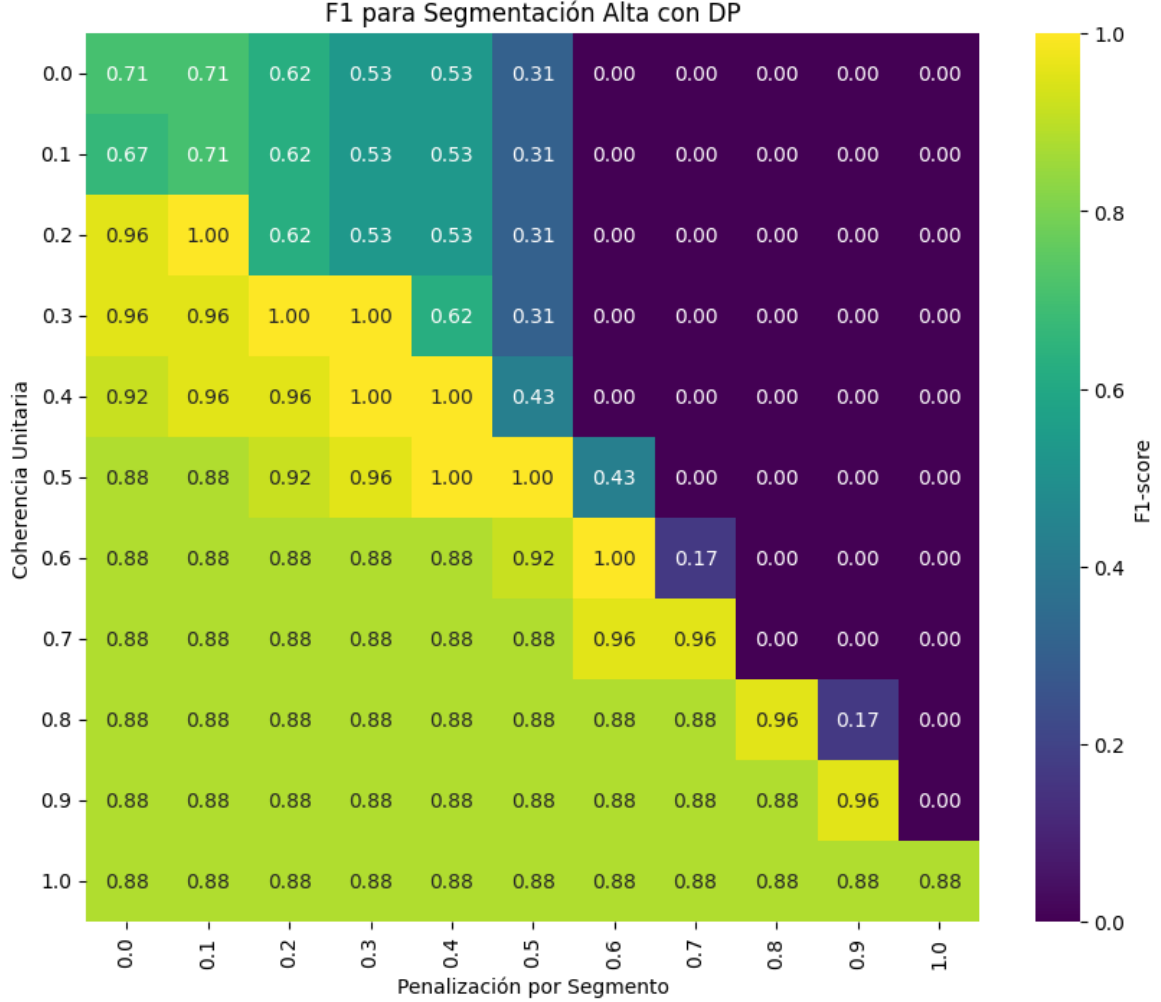


Figura 3.1: Mapa de calor del F_1 para la instancia de segmentación alta.

La Figura 3.1 permite distinguir tres comportamientos en función de los parámetros:

- **Diagonal (ϕ_u moderado y α equilibrado):** la solución devuelta coincide exactamente con la segmentación de referencia ($F_1 \approx 1$).
- **Región inferior (ϕ_u alto, α bajo):** la escasa penalización por segmento y la alta coherencia unitaria favorecen la creación de segmentos de una sola frase, pero la solución no logra capturar los segmentos multielemento presentes en el óptimo, reduciendo el valor de F_1 .
- **Región superior (ϕ_u bajo, α alto):** cuando $\alpha > 0,5$ la penalización domina sobre la coherencia y el algoritmo opta por la segmentación trivial sin cortes ($F_1 = 0$),

ya que cualquier segmentación intermedia queda suficientemente penalizada como para no compensar la ganancia en coherencia.

3.1.2. Caso 2: Segmentación media

La instancia posee un número moderado de cortes reales, combinando segmentos de longitud variada.

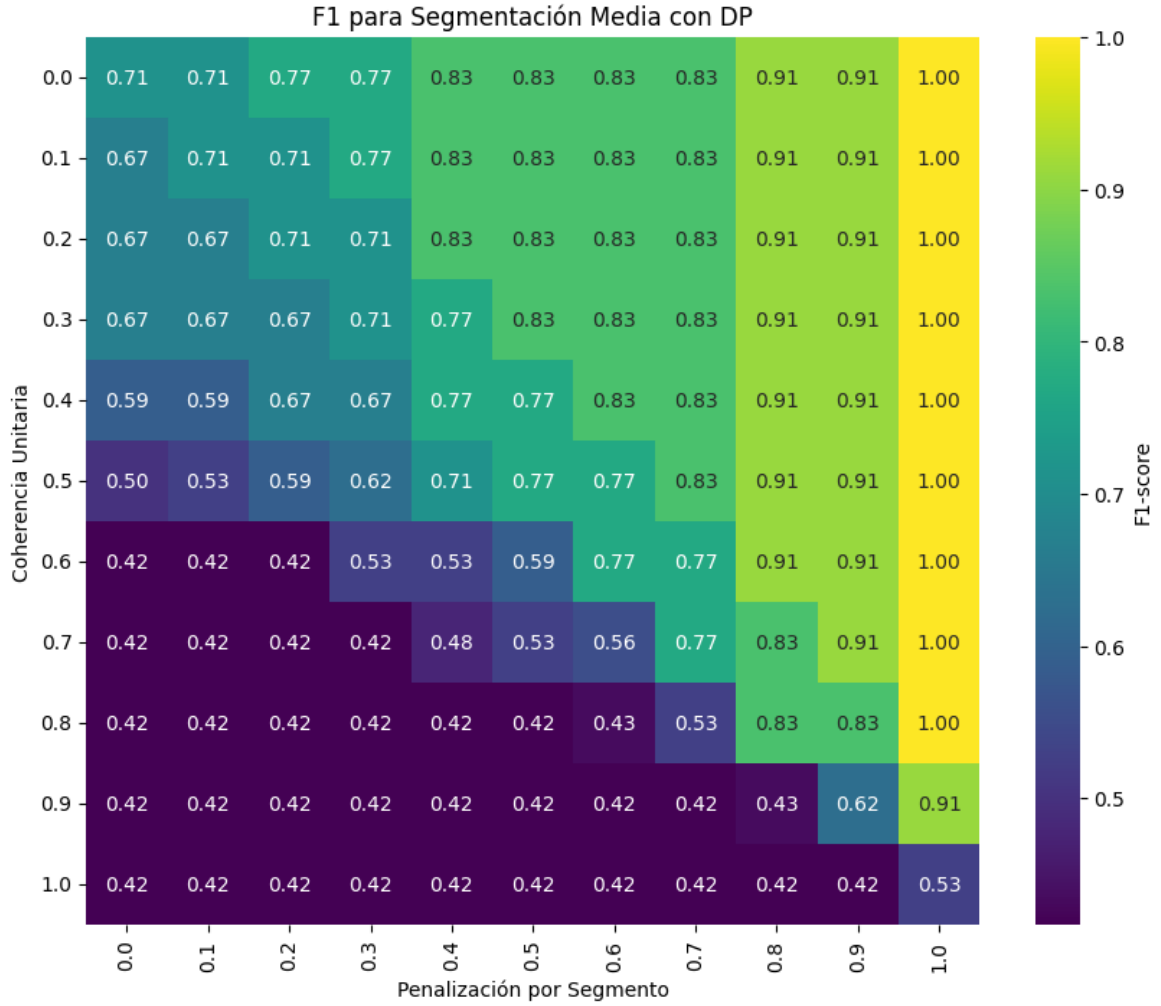


Figura 3.2: Mapa de calor del F_1 para la instancia de segmentación media.

La Figura 3.2 revela un patrón similar al caso anterior, aunque con algunas diferencias relevantes:

- **Región inferior (ϕ_u alto, α bajo):** la baja penalización conduce de nuevo a una segmentación excesiva, con cortes espurios que deterioran notablemente el F_1 .
- **Región superior (ϕ_u bajo, α moderado):** se obtienen resultados intermedios. La

penalización frena la sobre-segmentación, pero sin alcanzar la estructura exacta de referencia.

- **Columna derecha ($\alpha = 1,0$, penalización máxima):** la solución coincide con el óptimo ($F_1 = 1$) para prácticamente todos los valores de ϕ_u , excepto cuando ϕ_u es muy alto ($\phi_u > 0,8$), lo que demuestra que incluso con penalización máxima sigue siendo necesario ajustar la coherencia unitaria para evitar falsos segmentos unitarios.

3.1.3. Caso 3: Segmentación baja

Esta instancia contiene un único corte real, por lo que la segmentación óptima está formada por solo dos segmentos.

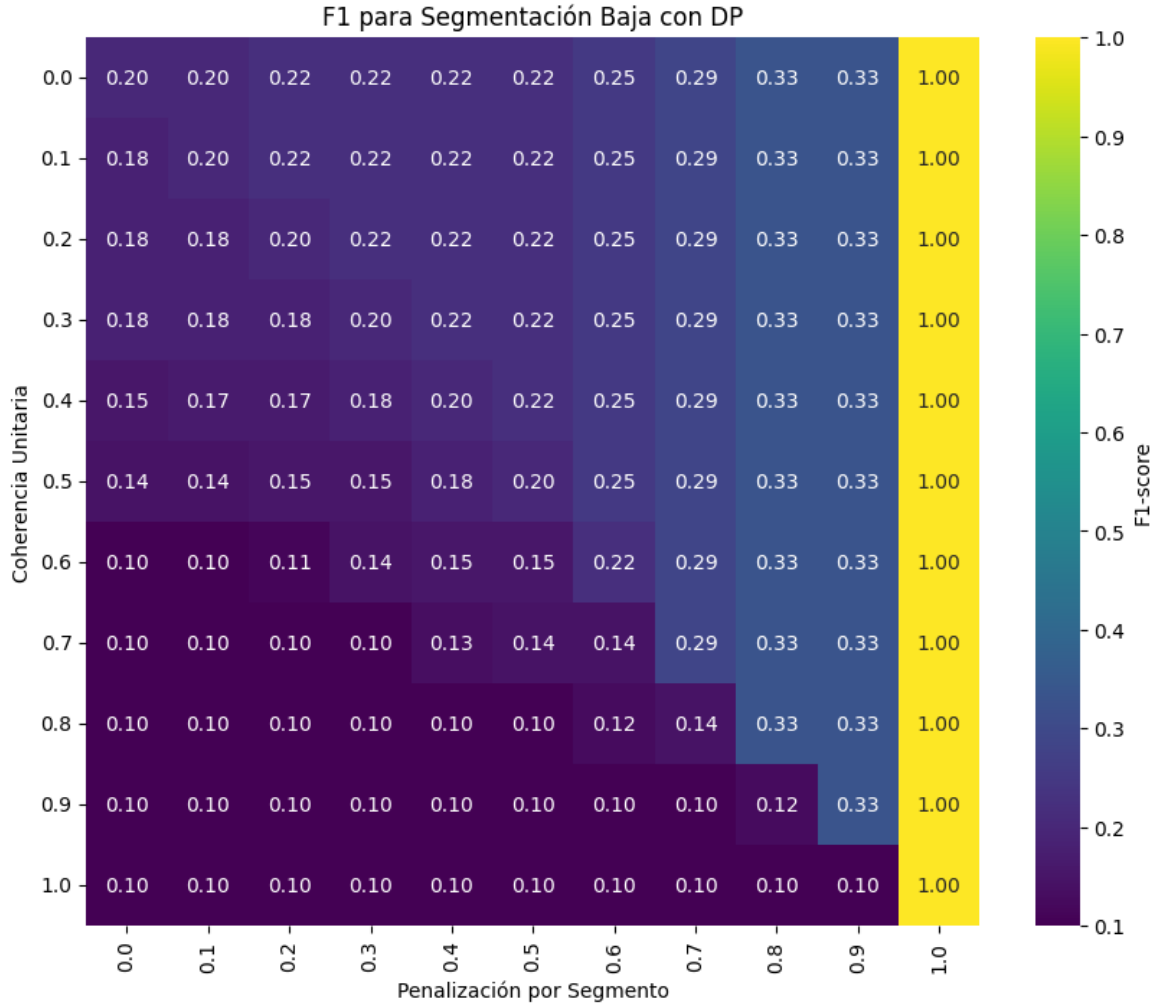


Figura 3.3: Mapa de calor del F_1 para la instancia de segmentación baja.

La Figura 3.3 muestra un comportamiento más extremo que en los casos anteriores:

- **Columna derecha** ($\alpha = 1,0$): la penalización máxima fuerza a que la solución tenga exactamente un corte, y el algoritmo acierta su ubicación correcta, alcanzando $F_1 = 1$ para todos los valores de ϕ_u . La coherencia unitaria resulta irrelevante porque no hay espacio para falsos segmentos unitarios.
- **Resto del mapa** ($\alpha < 1,0$): cualquier penalización inferior provoca una segmentación excesiva, añadiendo cortes inexistentes que alejan la solución de la referencia y producen valores de F_1 bajos o nulos. La presencia de un solo corte real hace que incluso pequeñas desviaciones sean severamente penalizadas por la métrica.

3.1.4. Análisis

Los mapas de calor ponen de manifiesto un conflicto esencial: ninguna combinación de parámetros permite resolver correctamente a la vez instancias muy fragmentadas (Caso 1) e instancias casi sin cortes (Caso 3). La única candidata que cubriría ambos extremos sería $\phi_u = \alpha = 1,0$, pero ese ajuste introduce falsos segmentos unitarios en el caso de segmentación media, degradando su F_1 de forma apreciable.

Ante esta situación, se prioriza el Caso 2 por ser el más representativo de una instancia promedio. Se fijan, por tanto, valores que maximizan su F_1 ($\phi_u = 0,8$ y $\alpha = 1,0$), aun a costa de no alcanzar el óptimo en el Caso 1.

Esta decisión se ve respaldada por la naturaleza esperable de las instancias reales. En aplicaciones con grandes volúmenes de datos textuales, los segmentos puramente unitarios rara vez constituyen una segmentación óptima; los cortes suelen agrupar varios elementos relacionados. Por tanto, la incapacidad de resolver el Caso 1 no compromete la utilidad práctica del modelo. Si en un escenario concreto se sospechara la presencia de segmentos unitarios relevantes, se recomendaría elevar ambos parámetros a 0,8, desplazando la solución hacia la diagonal que mostró mejores resultados en el Caso 1 sin renunciar por completo a la penalización.

Por último, conviene señalar una característica de la función objetivo resultante. Con $\alpha = 1,0$, la función de *fitness* es no positiva: el valor máximo teórico es 0, que se alcanzaría únicamente si cada segmento obtuviera una coherencia perfecta de 1 y esta anulara exactamente la penalización acumulada. La solución trivial sin cortes (un solo segmento) tiene como cota inferior -1 (coherencia total $\in [0, 1]$ menos una unidad de penalización). Esta estrechez del rango de valores, unida a la inexactitud inherente a la estimación de coherencia mediante el LLM, podría favorecer sistemáticamente soluciones subóptimas con pocos cortes, ya que pequeñas fluctuaciones en la puntuación de coherencia pueden hacer que la penalización domine y que el algoritmo se estanque. A pesar de ello, este experimento mostró que $\alpha = 1,0$ es la única penalización capaz de contrarrestar la tendencia a la sobre-segmentación, por lo que se mantiene como elección de compromiso.

3.2 Experimento 2: Ajuste de las metaheurísticas

Objetivo. Encontrar la configuración de hiperparámetros que maximice el rendimiento de cada metaheurística sobre el problema de segmentación.

Metodología. Para cada algoritmo se muestrearon aleatoriamente 10 combinaciones de hiperparámetros dentro de rangos predefinidos. Cada configuración se evaluó sobre la instancia de validación promedio con 3 repeticiones. Se midió el F_1 medio respecto a la segmentación de referencia. A partir de los resultados se seleccionó la mejor configuración de cada algoritmo.

3.2.1. Ascenso de Colina

- **Rango explorado:**
 - Límite de estancamiento: {50, 100, 150, 200}
 - Evaluaciones máximas: {500, 1000, 5000}
 - Tamaño de vecindad: {1, 3, 5}
- **Muestreo y resultados:** Tabla 3.1.

Cuadro 3.1: Resultados del muestreo aleatorio para Ascenso de colina

Límite estanc.	Max eval.	Tamaño vecindad	F1 medio
150	500	5	0.8571
50	5000	1	0.7143
150	1000	5	1.0000
200	500	3	1.0000
200	1000	1	0.8571
150	500	3	1.0000
50	500	1	1.0000
100	1000	3	1.0000
50	500	5	0.6190
50	1000	5	1.0000

Varias combinaciones alcanzaron el óptimo $F_1 = 1$. La configuración seleccionada, que equilibra bajo coste y exploración suficiente, es:

- Límite de estancamiento: 150
- Evaluaciones máximas: 500
- Tamaño de vecindad: 3

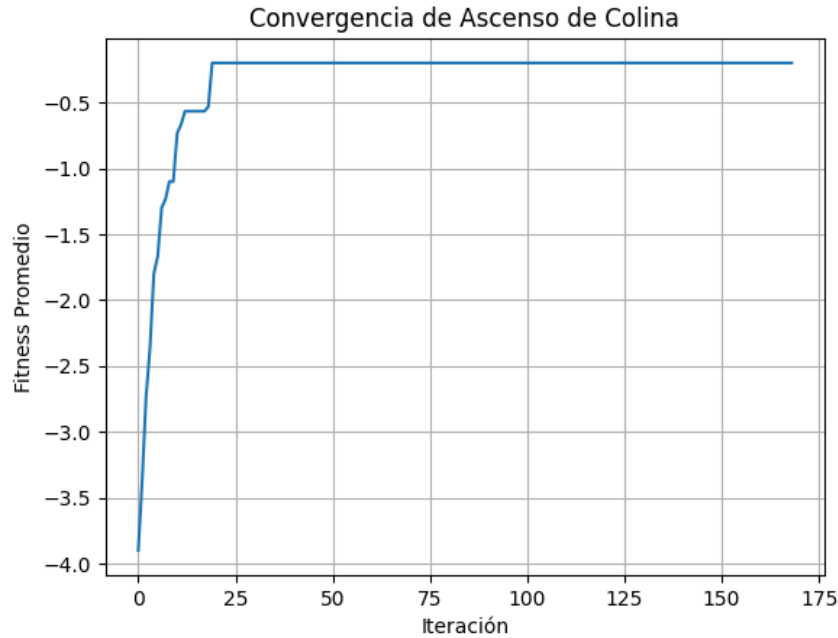


Figura 3.4: Convergencia del promedio del mejor fitness para la configuración seleccionada de Ascenso de Colina.

La vecindad de tamaño 3 proporciona suficiente diversidad local sin multiplicar innecesariamente las evaluaciones, mientras que el límite de estancamiento 150 da margen para escapar de óptimos locales poco profundos. El gráfico 3.4 muestra la rapidez con la que el algoritmo converge a un óptimo, lo que justifica la elección de solo 500 evaluaciones.

3.2.2. Recocido Simulado

■ **Rango explorado:**

- Temperatura inicial (T_0): {10, 50, 100}
- Factor de enfriamiento (α): {0,8, 0,9, 0,95, 0,99}
- Iteraciones por temperatura: {5, 10, 20, 50}
- Evaluaciones máximas: {1000, 2000, 5000}
- Temperatura mínima: 0,001 (fija)

■ **Muestreo y resultados:** Tabla 3.2.

Cuadro 3.2: Resultados del muestreo aleatorio para Recocido simulado

T0	Alfa	Iter./T	Max eval.	F1 medio
50	0.9	20	5000	0.3810
50	0.9	20	2000	0.5820
10	0.95	20	1000	0.4762
10	0.99	20	5000	0.4762
10	0.8	20	5000	0.4762
50	0.8	5	2000	0.6349
100	0.9	20	1000	0.4405
50	0.9	5	2000	0.8201
100	0.99	20	5000	0.4868
100	0.9	10	5000	0.4762

La combinación con mejor rendimiento fue:

- $T_0 = 50$
- $\alpha = 0,9$
- Iteraciones por temperatura: 5
- Evaluaciones máximas: 2000

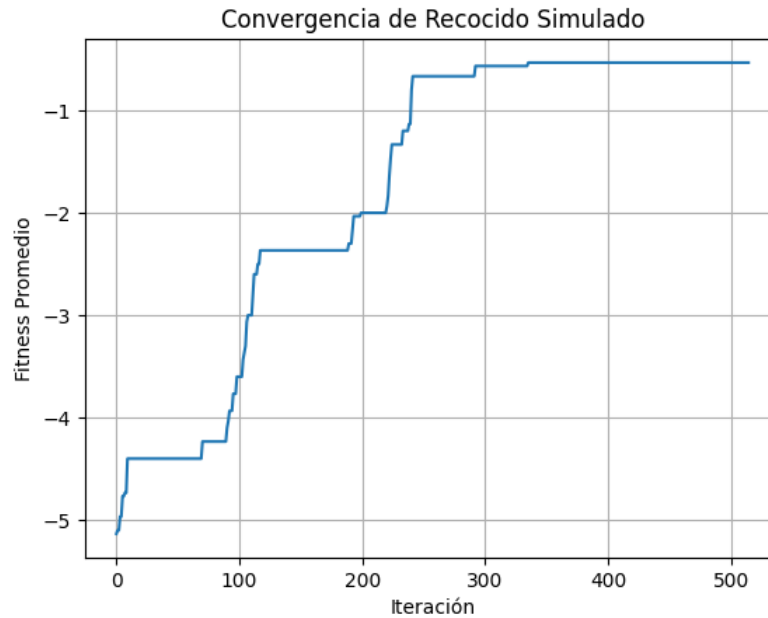


Figura 3.5: Convergencia del promedio del mejor fitness para la mejor configuración de Recocido Simulado.

El recocido simulado no obtuvo buenos resultados. La escala pequeña de la función de *fitness* hace que las diferencias Δf entre soluciones vecinas sean ínfimas; la probabilidad de aceptar empeoramientos permanece alta incluso con temperaturas moderadas, lo que dificulta la convergencia fina. La configuración con $\alpha = 0,9$ y pocas iteraciones por temperatura (5) resultó la más efectiva, ya que un enfriamiento más rápido reduce antes la aceptación de soluciones malas.

3.2.3. Algoritmo Genético

■ **Rango explorado:**

- Tamaño de población: {10, 20, 50}
- Tasa de mutación: {0,01, 0,05, 0,1}
- Elitismo: {1, 3, 5}
- Generaciones máximas: {20, 50, 100}
- Evaluaciones máximas: {1000, 5000}

■ **Muestreo y resultados:** Tabla 3.3.

Cuadro 3.3: Resultados del muestreo aleatorio para Algoritmo genético

Población	Tasa mut.	Elitismo	Gen.	Max eval.	F1 medio
50	0.05	1	100	5000	1.0000
10	0.10	5	50	1000	0.7778
10	0.05	5	50	5000	0.6778
50	0.05	1	20	5000	1.0000
50	0.10	3	100	5000	0.7619
20	0.01	3	100	5000	0.5464
50	0.05	3	100	1000	1.0000
20	0.10	5	20	5000	0.8333
20	0.10	5	50	1000	0.7619
10	0.05	3	20	5000	0.6086

La configuración con menor presupuesto que alcanzó $F_1 = 1$ es: población 50, tasa de mutación 0.05, elitismo 3, 100 generaciones, 1000 evaluaciones.

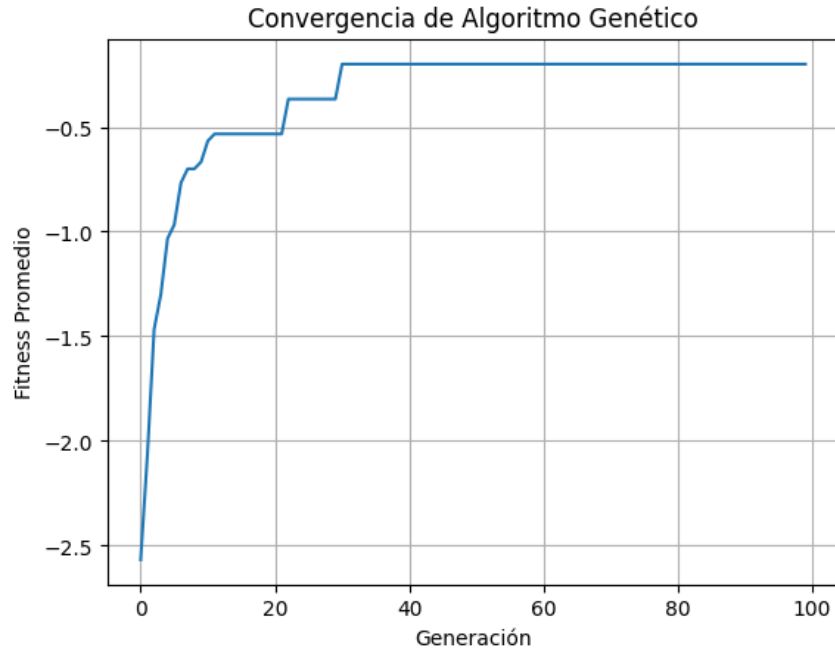


Figura 3.6: Convergencia del promedio del mejor fitness para la configuración seleccionada del Algoritmo genético.

La población grande y una tasa de mutación moderada proporcionan un equilibrio adecuado: suficiente diversidad para explorar sin destruir los bloques prometedores. En contraste, las configuraciones con población pequeña y mutación alta degradan el rendimiento, ya que la excesiva variación impide la consolidación de buenos esquemas.

3.2.4. Enjambre de Partículas

■ **Rango explorado:**

- Tamaño de población: {10, 20, 50}
- Inercia (w): {0,5, 0,7, 0,9}
- Coeficiente cognitivo (c_1): {1,0, 1,5, 2,0, 2,5}
- Coeficiente social (c_2): {1,0, 1,5, 2,0, 2,5}
- Generaciones máximas: {20, 50, 100}
- Evaluaciones máximas: {1000, 5000}
- Velocidad máxima ($v_{\text{máx}}$): {2,0, 5,0, 10,0}

■ **Muestreo y resultados:** Tabla 3.4.

Cuadro 3.4: Resultados del muestreo aleatorio para Enjambre de partículas (PSO)

Pobl.	w	c1	c2	Gen.	Max eval.	v_max	F1 medio
10	0.9	2.5	2.0	100	5000	2.0	0.9195
50	0.7	1.0	2.0	20	1000	10.0	0.9117
10	0.5	2.0	1.5	50	5000	2.0	0.8012
50	0.9	1.0	2.5	50	1000	2.0	0.8779
50	0.7	1.0	2.0	50	5000	2.0	0.9405
50	0.7	2.0	2.0	100	5000	5.0	1.0000
20	0.7	1.5	1.0	50	1000	5.0	0.8824
10	0.7	1.0	2.0	50	1000	5.0	0.8453
50	0.5	1.5	2.5	20	1000	5.0	0.8934
10	0.5	1.5	1.5	50	1000	2.0	0.8196

La única configuración que alcanzó $F_1 = 1$ fue: población 50, $w = 0,7$, $c_1 = 2,0$, $c_2 = 2,0$, 100 generaciones, 5000 evaluaciones, $v_{\text{máx}} = 5,0$.

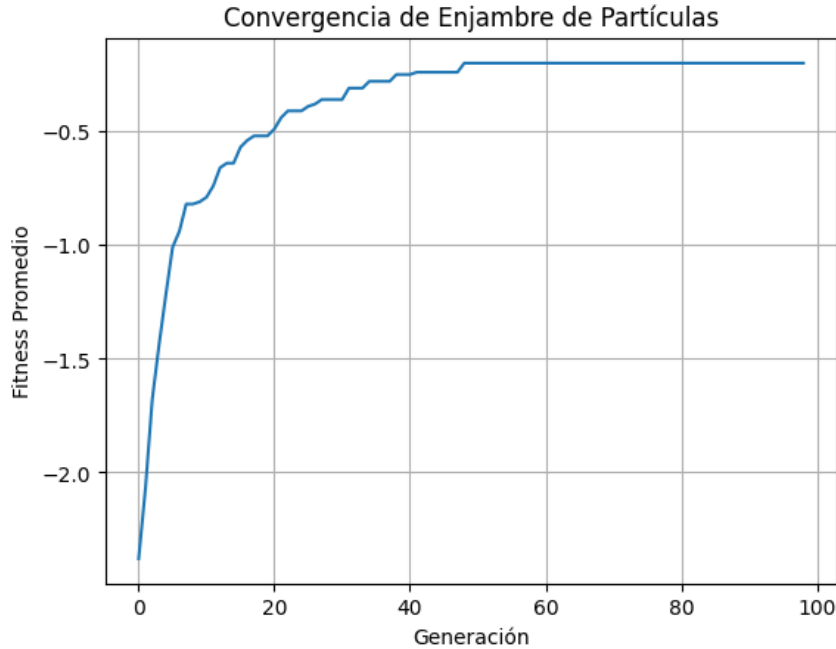


Figura 3.7: Convergencia del promedio del mejor fitness para la mejor configuración de PSO.

La inercia $w = 0,7$ y coeficientes $c_1 = c_2 = 2,0$ equilibran la exploración individual y la social. El límite de velocidad $v_{\text{máx}} = 5,0$ evita la saturación de la sigmoide, mientras que la población grande y 100 generaciones ofrecen el tiempo necesario para que el enjambre converja al óptimo global de forma precisa.

3.2.5. Análisis

Los resultados del ajuste ponen de manifiesto diferencias importantes entre las metaheurísticas:

- **Algoritmos que alcanzan el óptimo:** Ascenso de Colina, Algoritmo Genético y PSO consiguen $F_1 = 1$ en la instancia de validación. La elección de las configuraciones finales se basó en criterios de robustez y eficiencia computacional.
- **Consistencia de PSO:** todos los resultados de PSO se sitúan por encima de $F_1 = 0,8$, lo que demuestra una notable robustez frente a variaciones en los hiperparámetros. Incluso con poblaciones pequeñas (10 partículas) y distintas combinaciones de inercia y coeficientes, el algoritmo mantiene un rendimiento alto, confirmando su capacidad para explorar eficazmente el espacio de soluciones.
- **Sensibilidad y coste:** Ascenso de colina destaca por su bajo coste. Los algoritmos poblacionales requieren más recursos, pero ofrecen mecanismos de exploración más potentes que pueden ser decisivos en instancias más complejas.
- **Convergencia:** Las curvas muestran que Ascenso de Colina progresa rápidamente hacia el óptimo, mientras que GA y PSO mejoran gradualmente a lo largo de las generaciones. Recocido Simulado mejora el *fitness* de forma lenta y escalonada antes de estancarse en un óptimo local.

3.3 Experimento 3: Evaluación de las metaheurísticas

Objetivo. Comparar el rendimiento de las cuatro metaheurísticas, cada una con su mejor configuración y sobre un conjunto de instancias no vistas durante el ajuste.

Metodología. Se evaluó la mejor configuración de cada algoritmo seleccionada en el Experimento 2 ¹ sobre las cinco instancias de prueba, registrando la métrica F_1 y el *fitness*. Se reportan la media, desviación estándar, valores extremos y el número promedio de evaluaciones consumidas.

3.3.1. Resultados

La Tabla 3.5 resume los resultados. El Enjambre de Partículas (PSO) alcanzó el mejor F_1 medio (0,9025), seguido de cerca por el Algoritmo Genético (0,8814). El Ascenso de Colina obtuvo un rendimiento intermedio (0,7162) y el Recocido Simulado mostró la mayor variabilidad y el peor valor medio (0,5447).

¹Debido a limitaciones logísticas, se estableció 1000 como cota superior del número máximo de evaluaciones.

Cuadro 3.5: Resultados comparativos sobre las cinco instancias de prueba

Metaheurística	F1 medio	F1 std	F1 min	F1 max	Evaluaciones
Enjambre de partículas (PSO)	0.9025	0.0000	0.9025	0.9025	1000
Algoritmo genético	0.8814	0.0000	0.8814	0.8814	1037
Ascenso de colina	0.7162	0.0000	0.7162	0.7162	496
Recocido simulado	0.5447	0.1316	0.3578	0.7348	246

La Figura 3.8 muestra las curvas de convergencia promediadas durante las ejecuciones.

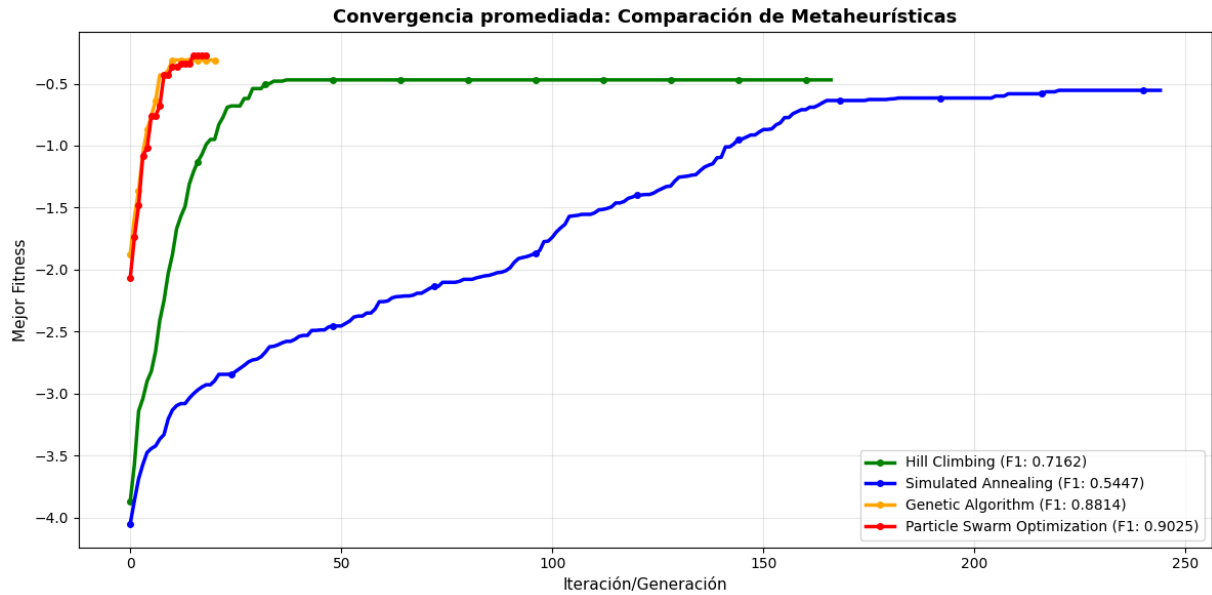


Figura 3.8: Comparación de las curvas de convergencia (mejor fitness) de las cuatro metaheurísticas sobre las instancias de prueba.

La observación de las curvas de convergencia y del consumo de evaluaciones revela diferencias en la dinámica de búsqueda:

- **Ascenso de Colina** se detiene tras consumir en promedio 496 evaluaciones de las 500 disponibles. El criterio de parada que actúa es el límite de estancamiento: la búsqueda local queda atrapada en un óptimo local y el algoritmo se detiene automáticamente al superar el umbral de iteraciones sin progreso.
- **Recocido Simulado** apenas utiliza 246 evaluaciones de las 1000 permitidas. Esta parada tan prematura evidencia un enfriamiento excesivamente rápido: la temperatura desciende por debajo del valor mínimo $T_{\min}$ antes de que el algoritmo haya podido explorar regiones alternativas del espacio de búsqueda. La alta variabilidad en sus resultados (F_1 entre 0.36 y 0.73) refleja la dependencia del azar en un esquema que no llega a converger adecuadamente.

- **Algoritmo genético y PSO** sí agotan su presupuesto de evaluaciones, pero el gráfico de convergencia revela que lo hacen tras haber completado pocas generaciones. Con un número muy limitado de ciclos evolutivos, ambos algoritmos consiguen concentrar la búsqueda en regiones de alta calidad y alcanzan soluciones buenas de forma consistente, lo que confirma su eficiencia y convierte estos resultados en un excelente punto de partida.

3.3.2. Análisis

El Enjambre de Partículas resultó ser la estrategia más efectiva para el problema de segmentación. Su buen desempeño concuerda con la consistencia observada en el Experimento 2, donde incluso las configuraciones menos favorables mantenían un rendimiento superior al de otras metaheurísticas.

Las diferencias observadas se explican por la estructura del paisaje de búsqueda. La función objetivo genera múltiples óptimos locales distribuidos por todo el espacio. Los algoritmos de trayectoria carecen de mecanismos para escapar de estas trampas una vez que se aproximan a ellas; el primero porque solo acepta mejoras, el segundo porque se enfría antes de poder saltar a otra cuenca de atracción. Los enfoques poblacionales, en cambio, mantienen una diversidad de soluciones que les permite explorar simultáneamente distintas regiones y recombinar información prometedora, lo que les confiere una ventaja decisiva en este tipo de paisajes.

En consecuencia, se selecciona el Enjambre de Partículas con su configuración óptima ($w = 0,7$, $c_1 = c_2 = 2,0$, población 50, $v_{\text{máx}} = 5,0$) como la estrategia final para el sistema de segmentación de contenido basado en coherencia semántica.

Capítulo 4

Conclusiones

1. **Formalización efectiva del problema.** La formulación mediante un vector binario de cortes, una función de coherencia agregada evaluada por el LLM y un término de penalización por número de segmentos (α) ha demostrado ser adecuada para capturar la esencia de la tarea. La introducción de una puntuación fija para segmentos unitarios (ϕ_u) permitió evitar llamadas innecesarias a la API y controlar la fragmentación excesiva.
2. **Interacción crítica entre parámetros de la función objetivo.** El estudio paramétrico con solución exacta (Experimento 1) reveló un conflicto inherente: ninguna combinación de ϕ_u y α optimiza simultáneamente todos los tipos de instancia. La elección de $\phi_u = 0,8$ y $\alpha = 1,0$ priorizó el caso promedio, reconociendo que en aplicaciones reales con textos extensos los segmentos puramente unitarios son poco frecuentes.
3. **Arquitectura de *framework* reutilizable y extensible.** Las clases abstractas `SingleSolutionMetaheuristic` y `PopulationMetaheuristic` proporcionan métodos `solve` universales que encapsulan el bucle de búsqueda completo y delegan en las subclases únicamente los operadores específicos de cada algoritmo. La implementación de Ascenso de Colina, Recocido Simulado, Algoritmo Genético y Enjambre de Partículas confirma la elegancia y flexibilidad del *framework*.
4. **Integración eficiente del modelo de lenguaje.** El uso de un *prompt* sencillo que solicita una puntuación numérica de coherencia, combinado con una caché persistente en disco, resultó esencial para amortizar el coste de las consultas a la API y permitir comparaciones justas entre algoritmos sobre evaluaciones idénticas.
5. **Superioridad de las metaheurísticas poblacionales.** En la evaluación final sobre instancias de prueba, el Enjambre de Partículas obtuvo el mejor rendimiento medio, seguido de cerca por el Algoritmo Genético. Ambos superaron ampliamente a los métodos de trayectoria, en particular a Recocido Simulado, que mostró el peor desempeño y alta variabilidad.

6. **Robustez y eficiencia del PSO.** La consistencia del PSO en todas las configuraciones probadas (Experimento 2) y su capacidad para alcanzar soluciones de alta calidad con un presupuesto limitado de 1000 evaluaciones lo convierten en la estrategia recomendada para el sistema final.

Capítulo 5

Recomendaciones

A partir de los resultados obtenidos y las limitaciones encontradas, se proponen las siguientes líneas de trabajo futuro y mejoras:

- **Ampliación del conjunto de datos y del presupuesto de evaluaciones.** El tamaño reducido del *dataset* (8 instancias de 20 elementos) y el límite tan reducido de evaluaciones impuesto en los experimentos restringen la generalización de las conclusiones y no permiten observar el verdadero potencial de las metaheurísticas. Se recomienda construir un *benchmark* más extenso, con secuencias de 50 a 200 elementos, diversos dominios temáticos y anotaciones consensuadas por múltiples jueces humanos, y repetir la comparación con un presupuesto de consultas sustancialmente mayor (del orden de 10 000 o 50 000 evaluaciones). Esto permitiría validar la escalabilidad de los algoritmos y afinar los parámetros con mayor confianza estadística.
- **Exploración de funciones objetivo alternativas.** La penalización lineal $\alpha \cdot k$ demostró ser efectiva pero generó un rango de *fitness* muy estrecho que perjudicó al Recocido Simulado. Se sugiere investigar penalizaciones no lineales, normalizaciones adaptativas o funciones multiobjetivo que optimicen simultáneamente la coherencia media y el número de segmentos, proporcionando un paisaje de búsqueda más amigable para algoritmos basados en diferencias de calidad.
- **Incorporación de modelos de lenguaje alternativos y estudio del *prompt*.** El uso de `gemin-3.1-flash-lite` respondió a un criterio práctico, pero conviene evaluar el impacto de modelos con mayor capacidad semántica o ajustados mediante *fine-tuning* en la calidad de las puntuaciones de coherencia. De forma complementaria, se recomienda experimentar con distintos diseños del *prompt* (escala de puntuación más granular, petición de justificación previa, inclusión de ejemplos) para analizar cómo afectan al rendimiento final del sistema.

Bibliografia

- [1] Sylvain Lamprier, Tassadit Amghar, Bernard Levrat, and Frédéric Saubion. Toward a more global and coherent segmentation of texts. *Applied Artificial Intelligence*, 22(3):208–234, 2008.
- [2] El-Ghazali Talbi. *Metaheuristics: from design to implementation*. John Wiley & Sons, 2009.